PART 5 OF 8 · SOLUTIONS

Stochastic Calculus, Step by Step

Part 5: the maths toolkit.
Solutions to every exercise



Rates with Bip

STOCHASTIC CALCULUS, STEP BY STEP · PART 5 OF 8 · SOLUTIONS

Stochastic Calculus, Step by Step

Part 5: The maths toolkit. Solutions

Rates with Bip

Stochastic Calculus, Step by Step. Part 5: The maths toolkit. Solutions.

© 2026 Rates with Bip. This book is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International licence (CC BY-NC-ND 4.0): you may copy and share it unchanged, for non-commercial purposes, crediting Rates with Bip. Licence: creativecommons.org/licenses/by-nc-nd/4.0. The companion notebooks are free under the MIT licence at github.com/RatesWithBip/curriculum.

The course's numbers: $S_0 = 100$, $K = 105$, $r = 3\%$ (continuously compounded), $\sigma = 20\%$, $T = 1$ year; Lesson 27's second share and Lesson 28's Vasicek rate where a lesson needs them. Made up for teaching; every random number is simulated with a fixed seed.

This volume holds the full solutions to the exercises and self-test questions in Stochastic Calculus, Step by Step, Part 5. Try each exercise before reading its solution.

Every number in the solutions is computed by a script that checks itself with assertions; the code solutions are shown in full.

This book is for education only. It is not financial, investment, legal or tax advice.

Contents

How to use the solutions 5

31. Linear Algebra for Several Risks 6

32. Ordinary Differential Equations 12

33. Complex Numbers and Fourier Transforms 18

34. Numerical Methods: Roots, Interpolation, Integration, Fitting 24

35. Regression and PDEs, Further 31

How to use the solutions

This volume holds the full solutions to *Stochastic Calculus, Step by Step, Part 5: The maths toolkit*: every exercise worked out, and the answers to the self-test questions at the end of each lesson. It is meant to be read **after** you have tried each one.

- **By-hand exercises** (the first six of each lesson) are worked in full. Each solution starts with **the idea** (the one thought that unlocks it), then goes through numbered steps with the reason for each, and ends with a check or an interpretation. If you are stuck, read only the idea and try again before reading the steps.
- **Code exercises** (the last two of each lesson) show a complete Python solution and exactly what it prints. Every number in this volume is produced by a script that checks itself with assertions, using fixed random seeds, so your own run should match to the digits shown when you use the same seeds.
- **Self-test answers** are short: a sentence or two, and a number where there is one.

The course's numbers are the same throughout: $S_0 = 100$, $K = 105$, $r = 3\%$ (continuously compounded), $\sigma = 20\%$, $T = 1$ year, and the call is worth 7.1281. Lesson 27's second share ($S_0 = 95$, $\sigma = 30\%$, correlation 0.5) and Lesson 28's Vasicek rate ($\kappa = 0.5$, $\bar{r} = 4\%$, $s = 1\%$, $r_0 = 3\%$) appear where a lesson needs them.

Almost every by-hand solution in Part 5 uses one of three moves: **expand and collect**, whether the entries of $LL^\top$, the terms of a Taylor series or the powers of r in a bond equation; **multiply by the right thing**, an integrating factor, a conjugate or a test function, so that what's left is a single derivative or a real number; or **check against a case you know**, a normal distribution, a constant coefficient or a straight line. If your answer differs, check the usual suspects: a missing transpose, a sign in the integrating factor, e^{-iuk} written as e^{iuk}, and a forgotten factor of t in a forward rate.

This volume is for education only and is not financial advice.

SOLUTIONS TO LESSON 31

Linear Algebra for Several Risks

Solutions to Exercises 31.1 to 31.8 and the Lesson 31 self-test.

Exercise 31.1

The idea. Expand the variance of a sum (Lesson 3), then minimise a quadratic in one weight.

Step 1. Half in each. With $w = (0.5, 0.5)$:

$$\begin{aligned}\text{Var} &= 0.5^2(0.2)^2 + 0.5^2(0.3)^2 + 2(0.5)(0.5)(0.5)(0.2)(0.3) \\ &= 0.01 + 0.0225 + 0.015 = 0.0475,\end{aligned}$$

a volatility of 0.2179.

Step 2. Any mix. With w in the first share and $1 - w$ in the second, and $\text{Cov} = \rho\sigma_1\sigma_2 = 0.03$:

$$V(w) = w^2\sigma_1^2 + (1-w)^2\sigma_2^2 + 2w(1-w)\rho\sigma_1\sigma_2.$$

Step 3. Set the derivative to zero.

$$\begin{aligned}V'(w) &= 2w\sigma_1^2 - 2(1-w)\sigma_2^2 + 2(1-2w)\rho\sigma_1\sigma_2 = 0 \\ \Longrightarrow \quad w &= \frac{\sigma_2^2 - \rho\sigma_1\sigma_2}{\sigma_1^2 + \sigma_2^2 - 2\rho\sigma_1\sigma_2}.\end{aligned}$$

Step 4. In numbers. $w = (0.09 - 0.03)/(0.04 + 0.09 - 0.06) = 0.06/0.07 = 0.8571$, and 0.1429 in the second share. The volatility is then 0.1964: below 20%, the volatility of the safer share alone. Adding a little of a riskier share lowers the risk, because the two don't move perfectly together.

Exercise 31.2

The idea. Write each X_i as a sum of the Z's, and use $E[Z_k Z_l] = 1$ if $k = l$ and 0 otherwise.

Covariance. $X_i = \sum_k L_{ik} Z_k$ has mean 0, so

$$\mathrm{Cov}(X_i, X_j) = E[X_i X_j] = \sum_k \sum_l L_{ik} L_{jl} E[Z_k Z_l] = \sum_k L_{ik} L_{jk} = (LL^\top)_{ij}.$$

The factor. With $L = \begin{pmatrix} a & 0 \\ b & c \end{pmatrix}$, $LL^\top = \begin{pmatrix} a^2 & ab \\ ab & b^2 + c^2 \end{pmatrix}$. Matching with Σ:

- $a^2 = 0.04$: $a = 0.20$;
- $ab = 0.03$: $b = 0.15$;
- $b^2 + c^2 = 0.09$: $c^2 = 0.09 - 0.0225 = 0.0675$, $c = 0.2598$.

Check. $LL^\top = \begin{pmatrix} 0.04 & 0.2 \times 0.15 \\ 0.15 \times 0.2 & 0.0225 + 0.0675 \end{pmatrix} = \begin{pmatrix} 0.04 & 0.03 \\ 0.03 & 0.09 \end{pmatrix}$. ✓

Exercise 31.3

The idea. One portfolio with negative variance is enough to rule a table out. For the full range, use the determinant.

The portfolio. $w^\top C w = \sum_i w_i^2 + 2\sum_{i<j} w_i w_j \rho_{ij}$. With $w = (1, -1, 1)$: $w_1 w_2 = -1$, $w_1 w_3 = 1$, $w_2 w_3 = -1$, so

$$w^\top C w = 3 + 2\big(-0.9 + (-0.9) - 0.9\big) = 3 - 5.4 = -2.4 < 0.$$

The range. Write $c = \rho_{13}$ and $a = 0.9$. A 3×3 correlation matrix is valid when its determinant is ≥ 0 (its 2×2 corners, $1 - a^2$ and $1 - c^2$, are already positive here). Expanding,

$$\det \begin{pmatrix} 1 & a & c \\ a & 1 & a \\ c & a & 1 \end{pmatrix} = 1 + 2a^2 c - 2a^2 - c^2.$$

With $a^2 = 0.81$: $1 + 1.62c - 1.62 - c^2 \geq 0$, that is $c^2 - 1.62c + 0.62 \leq 0$, or $(c - 0.62)(c - 1) \leq 0$. So the table is possible exactly when **$0.62 \leq \rho_{13} \leq 1$**. If 1 moves with 2 at 0.9 and 2 moves with 3 at 0.9, then 1 and 3 must be correlated at least 0.62. At $c = -0.9$ the determinant is -2.888.

Exercise 31.4

The idea. Write $C = (1 - \rho) I + \rho\, \mathbf{1}\mathbf{1}^\top$, where $\mathbf{1}$ is the vector of ones.

The vector of ones. $\mathbf{1}^\top \mathbf{1} = n$, so $C\mathbf{1} = (1 - \rho)\mathbf{1} + \rho\, \mathbf{1}\, n = \big(1 + (n-1)\rho\big)\mathbf{1}$. Eigenvalue $1 + (n-1)\rho$.

Vectors adding up to zero. If $\mathbf{1}^\top v = 0$, then $Cv = (1 - \rho)v + \rho\, \mathbf{1}\,(\mathbf{1}^\top v) = (1 - \rho)v$. Eigenvalue $1 - \rho$. These vectors fill out the other $n - 1$ directions.

The smallest common correlation. All eigenvalues must be ≥ 0. $1-\rho \geq 0$ always holds for $\rho \leq 1$; the binding condition is $1+(n-1)\rho \geq 0$, so $\rho \geq -1/(n-1)$. Five shares can't all be more negatively correlated than -0.25: if each falls when the others rise, they can't all do it at once. With $\rho = 0.5$, the eigenvalues are 3 and 0.5.

Exercise 31.5

The idea. Column by column: the diagonal entry from the leftover variance, then the entries below it from the leftover covariances.

Column 1. $L_{11} = \sqrt{1} = 1$; $L_{21} = 0.5/1 = 0.5$; $L_{31} = 0.3/1 = 0.3$.

Column 2. $L_{22} = \sqrt{1-0.5^2} = \sqrt{0.75} = 0.8660$. $L_{32} = (\rho_{23} - L_{31}L_{21})/L_{22} = (0.4 - 0.15)/0.8660 = 0.2887$.

Column 3. $L_{33} = \sqrt{1-0.3^2-0.2887^2} = \sqrt{1-0.09-0.0833} = 0.9092$.

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0.5 & 0.8660 & 0 \\ 0.3 & 0.2887 & 0.9092 \end{pmatrix}.$$

Reading it. Share 3 takes 0.3 of share 1's noise, 0.2887 of the new noise in share 2, and 0.9092 of its own: the squares add up to 1, as a correlation matrix's diagonal must.

Exercise 31.6

The idea. $w^\top \Sigma w$ with every weight $1/n$: n diagonal terms and $n(n-1)$ off-diagonal ones.

Step 1.

$$\text{Var} = \frac{1}{n^2}\left(n\sigma^2 + n(n-1)\rho\sigma^2\right) = \sigma^2\,\frac{1+(n-1)\rho}{n}.$$

Step 2. In numbers. $n=5$, $\rho=0.5$: $\sigma\sqrt{3/5} = 0.1549$.

Step 3. The limit. $\frac{1+(n-1)\rho}{n} = \frac{1}{n} + \rho\frac{n-1}{n} \to \rho$, so the volatility tends to $\sigma\sqrt{\rho} = 0.1414$.

Meaning. Diversification removes the part of each share's risk that is its own, but not the part that all the shares have in common. With correlation 0.5, no basket of these shares, however large, has volatility below 14.1%.

Exercise 31.7

Cholesky with a pivot check, tested against numpy.

```
import numpy as np


def cholesky(A):
    # Returns L with L @ L.T = A, or raises with the failing pivot.
    n = len(A)
    L = np.zeros((n, n))
    for j in range(n):
        d = A[j, j] - L[j, :j] @ L[j, :j]
        if d <= 0:
            raise ValueError(f"pivot {j + 1} is {d:.4f}: not positive definite")
        L[j, j] = math.sqrt(d)
        L[j + 1:, j] = (A[j + 1:, j] - L[j + 1:, :j] @ L[j, :j]) / L[j, j]
    return L


tests = {"two shares": np.array([[0.04, 0.03], [0.03, 0.09]]),
         "three shares": np.array([[1.0, 0.5, 0.3], [0.5, 1.0, 0.4], [0.3, 0.4, 1.0]]),
         "impossible table": np.array([[1.0, 0.9, -0.9], [0.9, 1.0, 0.9], [-0.9, 0.9, 1.0]])}
for name, A in tests.items():
    try:
        L = cholesky(A)
        print(f"{name:17s} ok, largest |LL' - A| = {np.abs(L @ L.T - A).max():.1e}")
    except ValueError as e:
        print(f"{name:17s} {e}")

rng = np.random.default_rng(317)
worst = 0.0
for _ in range(1000):
    G = rng.standard_normal((6, 12))
    C = np.corrcoef(G)                      # a random valid correlation matrix
    worst = max(worst, np.abs(cholesky(C) - np.linalg.cholesky(C)).max())
print(f"1000 random 6 x 6 correlation matrices: largest gap from numpy {worst:.1e}")
```

What it prints.

```
two shares        ok, largest |LL' - A| = 1.4e-17
three shares      ok, largest |LL' - A| = 1.1e-16
impossible table  pivot 3 is -15.2000: not positive definite
1000 random 6 x 6 correlation matrices: largest gap from numpy 7.2e-16
```

Reading it. The hand-written routine agrees with numpy's to rounding error on 1000 random matrices. On the impossible table it stops at the third pivot, where the leftover variance $1 - L_{31}^2 - L_{32}^2$ is negative: shares 1 and 2 have used up more than all of share 3's variance.

Exercise 31.8

The basket call against a Black-Scholes call at the basket's volatility.

```
def bs_call(s, k, t, vol):
    d1 = (math.log(s / k) + (r + 0.5 * vol**2) * t) / (vol * math.sqrt(t))
    return s * N(d1) - k * math.exp(-r * t) * N(d1 - vol * math.sqrt(t))


rng = np.random.default_rng(318)
Z = rng.standard_normal((400_000, 5))
print("  rho   basket (MC)     se    vol    BS at that vol    gap")
res = {}
for rho_ in (0.0, 0.25, 0.5, 0.75, 1.0):
    C = np.full((5, 5), rho_) + (1 - rho_) * np.eye(5)
    if rho_ < 1:
        X = Z @ cholesky(C).T
    else:
        X = np.repeat(Z[:, :1], 5, axis=1)  # perfectly correlated: one draw for all
    S = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * X)
    pay = math.exp(-r * T) * np.maximum(S.mean(axis=1) - K, 0)
    vol = sigma * math.sqrt((1 + 4 * rho_) / 5)
    approx = bs_call(S0, K, T, vol)
    res[rho_] = (pay.mean(), approx)
    print(f"{rho_:5.2f}  {pay.mean():9.4f}  {pay.std() / math.sqrt(len(pay)):7.4f}
{vol:.4f}   {approx:9.4f}   {pay.mean() - approx:+.4f}")
```

What it prints.

```
  rho   basket (MC)     se    vol    BS at that vol    gap
 0.00     2.7644   0.0078  0.0894      2.7316   +0.0328
 0.25     4.2134   0.0116  0.1265      4.1983   +0.0151
 0.50     5.3350   0.0147  0.1549      5.3301   +0.0049
 0.75     6.2832   0.0174  0.1789      6.2857   -0.0025
 1.00     7.1120   0.0198  0.2000      7.1281   -0.0161
```

Reading it. The approximation is worst at $\rho = 0$, where it misses by 0.0328, and best between 0.5 and 0.75. An average of independent lognormal prices is not lognormal: its distribution is less skewed, and a lognormal with the same volatility puts too little weight on high outcomes. As ρ rises the five shares move more and more as one, and at $\rho = 1$ the basket *is* a single share with volatility 20%, so the Black-Scholes price is exact: the gap of -0.0161 is within one standard error of the simulation.

Self-test

1. Because $w^\top \Sigma w$ is the variance of the portfolio $w^\top R$, and no variance can be negative.

2. $L = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{pmatrix}$: so $X_2 = \rho Z_1 + \sqrt{1-\rho^2}\, Z_2$.

3. 3, with eigenvector $(1, 1, 1, 1, 1)/\sqrt{5}$: all five shares moving together, the "market". It carries 0.60 of the total variance.

SOLUTIONS TO LESSON 32

Ordinary Differential Equations

Solutions to Exercises 32.1 to 32.8 and the Lesson 32 self-test.

Exercise 32.1

The idea. Bring the m term to the left, and multiply by $e^{\kappa t}$ so that the left side is one derivative.

Step 1. $m' + \kappa m = \kappa \bar{r}$. Multiply by $e^{\kappa t}$: $(e^{\kappa t} m)' = \kappa \bar{r} e^{\kappa t}$.

Step 2. Integrate from 0 to t. $e^{\kappa t} m(t) - r_0 = \bar{r}(e^{\kappa t} - 1)$.

Step 3. Divide. $m(t) = \bar{r} + (r_0 - \bar{r}) e^{-\kappa t}$.

In numbers. $e^{-0.5} = 0.6065$ and $e^{-2.5} = 0.0821$, so $E[r_1] = 4\% - 1\% \times 0.6065 = 0.03393$ and $E[r_5] = 0.03918$.

Half the gap. $e^{-\kappa t} = \frac{1}{2}$ when $t = \ln 2 / \kappa = 1.386$ years. After that, half of what is left closes in each further 1.39 years.

Exercise 32.2

The idea. The same equation with $a = 2\kappa$ and a constant right-hand side s^2.

Step 1. $(e^{2\kappa t} v)' = s^2 e^{2\kappa t}$, so $e^{2\kappa t} v(t) = \frac{s^2}{2\kappa}(e^{2\kappa t} - 1)$ and

$$v(t) = \frac{s^2}{2\kappa}\left(1 - e^{-2\kappa t}\right).$$

Step 2. The long run. $v \to s^2/(2\kappa)$, a standard deviation of $s/\sqrt{2\kappa} = 0.01/1 = 0.0100$.

Step 3. At one year. $v(1) = 0.0001 \times (1 - e^{-1}) =$ 6.3e-05, a standard deviation of 0.00795.

Meaning. Without the pull back to $\bar{r}$ ($\kappa = 0$), the variance would be $s^2 t$ and grow for ever. With it, new randomness is added at rate s^2 and old randomness decays at rate 2κ, and the two balance.

Exercise 32.3

The idea. Substitute the affine guess into Lesson 28's PDE and separate the terms in r from the rest.

Step 1. The PDE is $P_t + \kappa(\bar{r} - r)P_r + \frac{1}{2}s^2 P_{rr} = rP$. With $P = e^{A(\tau) - B(\tau)r}$ and $\tau = T - t$: $P_t = -(A' - B'r)P$, $P_r = -BP$, $P_{rr} = B^2 P$.

Step 2. Divide by P. $-A' + B'r - \kappa(\bar{r} - r)B + \frac{1}{2}s^2 B^2 = r$.

Step 3. Collect. Terms with r: $B' + \kappa B = 1$. Terms without: $-A' - \kappa\bar{r}B + \frac{1}{2}s^2 B^2 = 0$, that is $A' = -\kappa\bar{r}B + \frac{1}{2}s^2 B^2$. At maturity the bond pays 1 whatever r is, so $A(0) = B(0) = 0$.

Step 4. Solve for B. $(e^{\kappa\tau}B)' = e^{\kappa\tau}$, so $e^{\kappa\tau}B = (e^{\kappa\tau} - 1)/\kappa$ and $B(\tau) = (1 - e^{-\kappa\tau})/\kappa$. As $\tau \to \infty$, $B \to 1/\kappa = 2$; at 10 years it is already 1.9865.

Why the s^2 goes to A. In Vasicek the volatility s doesn't depend on r, so $\frac{1}{2}s^2 B^2$ has no r in it and lands in A's equation. In CIR it multiplies r, and lands in B's (Exercise 32.4).

Exercise 32.4

The idea. Differentiate the substitution and watch the squares cancel.

Step 1. With $y = -u'/(cu)$:

$$y' = -\frac{u''}{cu} + \frac{(u')^2}{cu^2}, \qquad cy^2 = \frac{(u')^2}{cu^2}.$$

Step 2. Substitute into $y' = a + by + cy^2$. The $(u')^2/(cu^2)$ terms appear on both sides and cancel, leaving $-\dfrac{u''}{cu} = a - \dfrac{bu'}{cu}$. Multiply by $-cu$:

$$u'' - bu' + acu = 0.$$

Step 3. The CIR case. $a = 1$, $b = -\kappa$, $c = -\frac{1}{2}s_c^2$ (with $s_c^2 = 0.003333$), so $u'' + \kappa u' - \frac{1}{2}s_c^2 u = 0$. Trying $u = e^{\lambda\tau}$:

$$\lambda^2 + \kappa\lambda - \tfrac{1}{2}s_c^2 = 0, \qquad \lambda = \tfrac{1}{2}\big(-\kappa \pm \gamma\big), \quad \gamma = \sqrt{\kappa^2 + 2s_c^2} = 0.506623.$$

The two exponents are 0.003311 and -0.503311.

Step 4. The long run. For large τ the term with the larger exponent dominates u, so $u'/u \to \lambda_+$ and $B = 2u'/(s_c^2 u) \to 2\lambda_+/s_c^2 = (\gamma - \kappa)/s_c^2 = 1.9868$. The same number is where $B' = 0$: the positive root of $1 - \kappa B - \frac{1}{2}s_c^2 B^2 = 0$.

Exercise 32.5

The idea. Separate the variables: everything with y on one side, dt on the other.

$y' = y^2$, $y(0) = 1$. $dy/y^2 = dt$, so $-1/y = t + C$; at $t = 0$, $C = -1$. So $y = 1/(1 - t)$, which blows up at $t = 1$.

$y' = 1 + y^2$, $y(0) = 0$. $dy/(1 + y^2) = dt$, so $\arctan y = t$ and $y = \tan t$, which blows up at $t = \pi/2 = 1.5708$.

$y' = 1 - y^2$, $y(0) = 0$. $dy/(1 - y^2) = dt$, so $\frac{1}{2}\ln\frac{1+y}{1-y} = t$ and $y = \tanh t$, which rises to 1 and never reaches it.

Why the sign decides. With $+y^2$, the bigger y gets, the faster it grows: growth feeds itself faster than exponentially, and y behaves like $1/(t^* - t)$ near a finite time t^*. With $-y^2$, the square acts as a brake: $y' = 0$ at $y = 1$, a level the solution approaches but can't cross.

Exercise 32.6

The idea. Each Euler step multiplies by the same factor.

Step 1. $y_{n+1} = y_n + h(-\kappa y_n) = (1 - \kappa h)y_n$, so $y_n = (1 - \kappa h)^n$.

Step 2. In numbers. $0.95^{10} = 0.598737$, against $e^{-0.5} = 0.606531$: an error of 0.00779. Halving the step: $0.975^{20} = 0.602688$, error 0.00384. The ratio of the errors is 2.03.

Step 3. Why the error is proportional to h. With $n = t/h$, $\ln y_n = \frac{t}{h}\ln(1 - \kappa h) = \frac{t}{h}\big(-\kappa h - \frac{1}{2}\kappa^2 h^2 - \cdots\big) = -\kappa t - \frac{1}{2}\kappa^2 t h - \cdots$. So $y_n \approx e^{-\kappa t}\big(1 - \frac{1}{2}\kappa^2 t h\big)$: an error of $\frac{1}{2}\kappa^2 t\, h\, e^{-\kappa t}$, proportional to h. For $t = 1$, $h = 0.1$: $0.0125 \times 0.6065 = 0.0076$, close to the measured 0.0078.

Step 4. Too big a step. If $h > 2/\kappa = 4$, then $|1 - \kappa h| > 1$: the Euler values grow, flipping sign each step, while the true solution decays. The same instability as Lesson 29's explicit scheme.

Exercise 32.7

The CIR yield curve by Runge-Kutta 4, against the closed form.

```
import numpy as np


def closed_form(tau):
    e = math.exp(gam * tau) - 1
    B = 2 * e / ((gam + kappa) * e + 2 * gam)
    A = (2 * kappa * rbar / sc**2) * math.log(2 * gam * math.exp((gam + kappa) * tau / 2) /
((gam + kappa) * e + 2 * gam))
    return A, B


def f(y):
    return np.array([-kappa * rbar * y[1], 1 - kappa * y[1] - 0.5 * sc**2 * y[1] ** 2])


h = 0.1
y = np.zeros(2)
print(" years   RK4 yield %   closed form %   gap (bp)")
worst = 0.0
for n in range(1, 301):
    k1 = f(y); k2 = f(y + h / 2 * k1); k3 = f(y + h / 2 * k2); k4 = f(y + h * k3)
    y = y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    tau = round(n * h, 10)
    if abs(tau - round(tau)) < 1e-9:
        A, B = closed_form(tau)
        y_rk = -(y[0] - y[1] * r0) / tau * 100
        y_cf = -(A - B * r0) / tau * 100
        worst = max(worst, abs(y_rk - y_cf) * 100)
        if round(tau) in (1, 2, 5, 10, 20, 30):
            print(f"{tau:6.0f}   {y_rk:10.6f}    {y_cf:10.6f}    {(y_rk - y_cf) * 100:+.2e}")
print(f"largest gap over 1 to 30 years: {worst:.1e} bp")
```

What it prints.

```
 years   RK4 yield %   closed form %   gap (bp)
     1     3.211850      3.211850    +3.03e-06
     2     3.364262      3.364262    +1.81e-06
     5     3.622063      3.622063    +3.61e-07
    10     3.784009      3.784009    -1.43e-09
    20     3.878252      3.878252    -1.77e-08
    30     3.910060      3.910060    -1.20e-08
largest gap over 1 to 30 years: 3.0e-06 bp
```

Reading it. With 300 steps of 0.1 year, the largest gap over 1 to 30 years is 3.0e-06 of a basis point: far below anything that matters. For a curve with no closed form, Runge-Kutta on the Riccati pair is as good as a formula.

Exercise 32.8

Step doubling on $y' = 1 + y^2$.

```
g = lambda y: 1 + y * y


def rk4_step(y, h):
    k1 = g(y); k2 = g(y + h / 2 * k1); k3 = g(y + h / 2 * k2); k4 = g(y + h * k3)
    return y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)


h = 0.01
y, t, first = 0.0, 0.0, None
print("    t        y      tan t    estimate")
while t < 1.57 - 1e-9:
    one = rk4_step(y, h)
    two = rk4_step(rk4_step(y, h / 2), h / 2)
    est = abs(one - two)
    y, t = two, round(t + h, 10)
    if first is None and est > 1e-6:
        first = t
    if abs(t * 100 - round(t * 100)) < 1e-9 and (round(t * 100) % 10 == 0 or t > 1.5):
        print(f"{t:5.2f}  {y:9.3f}  {math.tan(t):9.3f}  {est:.1e}")
print(f"the estimate first exceeds 1e-6 at t = {first:.2f}; the blow-up is at pi/2 =
{math.pi / 2:.4f}")
```

What it prints.

```
    t        y      tan t    estimate
 0.10      0.100      0.100  7.2e-13
 0.20      0.203      0.203  5.1e-13
 0.30      0.309      0.309  1.2e-13
 0.40      0.423      0.423  5.1e-13
 0.50      0.546      0.546  1.4e-12
 0.60      0.684      0.684  2.8e-12
 0.70      0.842      0.842  4.4e-12
 0.80      1.030      1.030  5.9e-12
 0.90      1.260      1.260  3.7e-12
 1.00      1.557      1.557  1.8e-11
 1.10      1.965      1.965  1.4e-10
 1.20      2.572      2.572  8.8e-10
 1.30      3.602      3.602  7.3e-09
 1.40      5.798      5.798  1.3e-07
 1.50     14.101     14.101  2.3e-05
 1.51     16.428     16.428  5.4e-05
 1.52     19.670     19.670  1.5e-04
 1.53     24.498     24.498  5.0e-04
 1.54     32.461     32.461  2.3e-03
 1.55     48.076     48.078  1.7e-02
 1.56     92.573     92.620  3.7e-01
 1.57    868.654   1255.766  2.8e+02
the estimate first exceeds 1e-6 at t = 1.45; the blow-up is at pi/2 = 1.5708
```

Reading it. The estimate first exceeds 10^{-6} at $t = 1.45$, about 0.12 before the blow-up, while the solution itself is still right to six digits. From there it grows by roughly a factor of 10 every 0.02. A solver that halves its step whenever the estimate is too big would take smaller and smaller steps and stall just short of $\pi/2$, which is exactly the warning wanted.

Self-test

1. e^{3t}. Then $(e^{3t}y)' = e^{3t}$, so $y = \frac{1}{3}(1 - e^{-3t})$.

2. In CIR the volatility is $s_c\sqrt{r}$, so the PDE's term $\frac{1}{2}s_c^2 r\, P_{rr} = \frac{1}{2}s_c^2 B^2\, r\, P$ contains r, and lands in the equation for B (the coefficient of r). In Vasicek, $\frac{1}{2}s^2B^2$ has no r and lands in A's equation.

3. By about 16 (in the lesson, 16.9): Runge-Kutta 4's error is proportional to h^4.

SOLUTIONS TO LESSON 33

Complex Numbers and Fourier Transforms

Solutions to Exercises 33.1 to 33.8 and the Lesson 33 self-test.

Exercise 33.1

The idea. Multiply out, replacing i^2 by -1; divide by multiplying top and bottom by the conjugate.

The product. $(3+4i)(1-2i) = 3 - 6i + 4i - 8i^2 = 3 - 2i + 8 = 11 - 2i$.

The modulus. $|3+4i| = \sqrt{9+16} = 5$.

The reciprocal. $$\frac{1}{3+4i} = \frac{3-4i}{(3+4i)(3-4i)} = \frac{3-4i}{25} = 0.12 - 0.16\text{i}.$$

Multiplying by i. $i(a+bi) = ai + bi^2 = -b + ai$: the point (a, b) moves to $(-b, a)$. Its length is $\sqrt{b^2+a^2}$, unchanged. And $(a,b)\cdot(-b,a) = -ab + ab = 0$: the two are at right angles. Taking $(1,0)$ to $(0,1)$ shows the turn is anticlockwise.

Exercise 33.2

The idea. Split the series for $e^{i\theta}$ by the powers of i, which cycle through $1, i, -1, -i$.

Euler's formula. In $e^{i\theta} = \sum_n (i\theta)^n/n!$, the even powers give $(-1)^k\theta^{2k}/(2k)!$, which is the series for $\cos\theta$, and the odd powers give $i(-1)^k\theta^{2k+1}/(2k+1)!$, which is i times the series for $\sin\theta$.

The modulus. The conjugate of $e^{i\theta}$ is $\cos\theta - i\sin\theta = e^{-i\theta}$, so $|e^{i\theta}|^2 = e^{i\theta}e^{-i\theta} = e^0 = 1$. (Equivalently, $\cos^2\theta + \sin^2\theta = 1$.)

The addition formulas. Multiply out $e^{ia}e^{ib} = (\cos a + i\sin a)(\cos b + i\sin b)$:

$$= (\cos a\cos b - \sin a\sin b) + i(\sin a\cos b + \cos a\sin b).$$

This equals $e^{i(a+b)} = \cos(a+b) + i\sin(a+b)$. Matching real and imaginary parts gives $\cos(a+b) = \cos a\cos b - \sin a\sin b$ and $\sin(a+b) = \sin a\cos b + \cos a\sin b$.

Exercise 33.3

The idea. Complete the square in the exponent, exactly as for the moment generating function.

Step 1.

$$E[e^{iuX}] = \int \frac{1}{\sqrt{2\pi s^2}} \exp\Big(-\frac{(x-m)^2}{2s^2} + iux \Big) dx.$$

Step 2. Complete the square.

$$-\frac{(x-m)^2}{2s^2} + iux = -\frac{(x-m-ius^2)^2}{2s^2} + ium - \tfrac{1}{2}s^2u^2.$$

(Expand the square on the right to check: the cross term gives $iux - ium$, and $-(ius^2)^2/(2s^2) = \frac{1}{2}s^2u^2$.)

Step 3. The factor $e^{ium - \frac{1}{2}s^2u^2}$ comes outside. What is left is a normal density shifted by ius^2, whose integral is 1, just as for a real shift. (Moving the line of integration by an imaginary amount needs a short argument from complex analysis; the answer can also be checked through the series $\sum (iu)^n E[X^n]/n!$.) So $\varphi(u) = e^{ium - \frac{1}{2}s^2u^2}$.

For $\ln S_T$. $m = \ln 100 + (0.03 - 0.02) \times 1 = 4.60517 + 0.01 = 4.61517$ and $s^2 = 0.04$:

$$\varphi(u) = \exp\big(4.61517\, iu - 0.02\, u^2\big).$$

Exercise 33.4

The idea. e^{iuX} lies on the unit circle.

- $\varphi(0) = E[e^0] = 1$.
- $|\varphi(u)| = |E[e^{iuX}]| \le E[|e^{iuX}|] = E[1] = 1$: the size of an average is at most the average size.
- $\varphi(-u) = E[e^{-iuX}] = E\big[\overline{e^{iuX}}\big] = \overline{E[e^{iuX}]} = \overline{\varphi(u)}$.

No moment generating function for S_T. $E[e^{tS_T}] = E\big[\exp(te^X)\big]$ with X normal. For large x, $\exp(te^x)$ grows like an exponential of an exponential, while the normal density falls only like $e^{-x^2/(2s^2)}$. The product grows without limit, so the integral is infinite for every $t > 0$. The characteristic function needs nothing of the kind: $|e^{iuX}| = 1$ whatever X is.

Exercise 33.5

The idea. Fold the weight e^X into the exponent: $e^{iuX}e^X = e^{i(u-i)X}$.

Step 1. Under the share measure, $E^S[g(X)] = E[g(X)e^X]/E[e^X]$. So

$$\varphi_S(u) = \frac{E[e^{iuX}e^X]}{E[e^X]} = \frac{E[e^{i(u-i)X}]}{E[e^{i(-i)X}]} = \frac{\varphi(u-i)}{\varphi(-i)},$$

since $i(u-i) = iu + 1$ and $i(-i) = 1$.

Step 2. For Black-Scholes. $\varphi(u-i) = \exp\left(i(u-i)m - \frac{1}{2}v(u-i)^2\right) = \exp\left(ium + m - \frac{1}{2}vu^2 + ivu + \frac{1}{2}v\right)$, and $\varphi(-i) = \exp(m + \frac{1}{2}v)$. Dividing:

$$\varphi_S(u) = \exp\left(iu(m+v) - \tfrac{1}{2}vu^2\right),$$

the characteristic function of a normal with the same variance and mean $m + v = \ln S_0 + (r + \frac{1}{2}\sigma^2)T = 4.65517$. That is Lesson 27's result: under the share measure the log price drifts at $r + \frac{1}{2}\sigma^2$, here 0.05. And $\varphi(-i) = E[S_T] = 103.0455$.

Exercise 33.6

The idea. Expand both factors to first order in u.

Step 1. $\varphi(u) = E[1 + iuX + O(u^2)] = 1 + iuE[X] + O(u^2)$, and $e^{-iuk} = 1 - iuk + O(u^2)$.

Step 2. Their product is $1 + iu(E[X] - k) + O(u^2)$. Divide by iu:

$$\frac{e^{-iuk}\varphi(u)}{iu} = \frac{1}{iu} + (E[X] - k) + O(u).$$

Step 3. $1/(iu) = -i/u$ is purely imaginary, so its real part is zero. The real part of the whole tends to $E[X] - k$.

In numbers. $E[\ln S_T] - \ln K = \ln(100/105) + 0.01 =$ -0.04879 + 0.01 = -0.03879. Negative, because the strike is above the average log price.

Exercise 33.7

Gil-Pelaez prices for eleven strikes, and the integration settings they need.

```
import numpy as np

v = sigma**2 * T
cf = lambda u: np.exp(1j * u * m - 0.5 * v * u**2)
cf_share = lambda u: cf(u - 1j) / cf(-1j)


def prob_above(phi, k, U, du):
    u = np.arange(du / 2, U, du)
    return 0.5 + (np.exp(-1j * u * k) * phi(u) / (1j * u)).real.sum() * du / math.pi


def fourier_call(k_strike, U, du):
    k = math.log(k_strike)
    return S0 * prob_above(cf_share, k, U, du) - k_strike * math.exp(-r * T) *
prob_above(cf, k, U, du)


def bs_call(k_strike):
    d1 = (math.log(S0 / k_strike) + (r + 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
    return S0 * N(d1) - k_strike * math.exp(-r * T) * N(d1 - sigma * math.sqrt(T))


strikes = range(80, 135, 5)
print("   U    du   worst error over strikes 80-130")
best = None
for U in (10, 15, 20, 25, 30):
    for du in (4.0, 0.5):
        worst = max(abs(fourier_call(k, U, du) - bs_call(k)) for k in strikes)
        print(f"{U:4d}  {du:4.1f}   {worst:.1e}")
        if best is None and worst < 5e-5:
            best = (U, du)
U, du = best
print(f"first (U, du) with four correct decimals at every strike: U = {U}, du = {du}")
print("strike   Fourier    formula")
for k in strikes:
    print(f"{k:6d}  {fourier_call(k, U, du):8.4f}  {bs_call(k):8.4f}")
```

What it prints.

```
   U    du   worst error over strikes 80-130
  10   4.0   7.9e-01
  10   0.5   3.6e-01
  15   4.0   8.9e-03
  15   0.5   2.3e-02
  20   4.0   3.4e-04
  20   0.5   5.2e-04
  25   4.0   6.9e-06
  25   0.5   4.4e-06
  30   4.0   7.8e-08
  30   0.5   1.5e-08
first (U, du) with four correct decimals at every strike: U = 25, du = 4.0
strike   Fourier    formula
    80   23.2240   23.2240
    85   19.1313   19.1313
    90   15.4292   15.4292
    95   12.1797   12.1797
   100    9.4134    9.4134
   105    7.1281    7.1281
   110    5.2934    5.2934
   115    3.8596    3.8596
   120    2.7666    2.7666
   125    1.9521    1.9521
   130    1.3577    1.3577
```

Reading it. The upper limit is what matters: $U = 25$ gives four correct decimals at every strike, even with the coarse step $du = 4$. The integrand falls off like $e^{-\frac{1}{2}\sigma^2 T u^2} = e^{-0.02u^2}$, about 4×10^{-6} at $u = 25$. With a smaller $\sigma\sqrt{T}$ (a shorter maturity, or a lower volatility) it falls off more slowly, and U must grow in proportion to $1/(\sigma\sqrt{T})$.

Exercise 33.8

A complex square root along a path, two ways.

```
u = np.linspace(0, 6 * math.pi, 1201)
zu = np.exp(1j * u) + 0.3
naive = np.exp(0.5 * np.log(zu))                          # principal log: angle in (-pi, pi]
smooth = np.sqrt(np.abs(zu)) * np.exp(0.5j * np.unwrap(np.angle(zu)))
jn = np.abs(np.diff(naive))
js = np.abs(np.diff(smooth))
print(f"principal log: {int((jn > 0.5).sum())} jumps, largest step {jn.max():.3f}")
print(f"unwrapped:     {int((js > 0.5).sum())} jumps, largest step {js.max():.4f}")
print("where the principal version jumps (u / pi):", np.round(u[1:][jn > 0.5] / math.pi,
3).tolist())
print(f"the two agree up to sign everywhere: max | |naive| - |smooth| | =
{np.abs(np.abs(naive) - np.abs(smooth)).max():.1e}")
```

What it prints.

```
principal log: 3 jumps, largest step 1.673
unwrapped:     0 jumps, largest step 0.0094
where the principal version jumps (u / pi): [1.0, 3.005, 5.0]
the two agree up to sign everywhere: max | |naive| - |smooth| | = 6.7e-16
```

Reading it. The principal version jumps 3 times, at $u = \pi$, 3π and 5π: exactly where $z(u) = e^{iu} + 0.3$ crosses the negative real axis (at $u = \pi$, $z = -0.7$). There its angle flips from π to $-\pi$, and the square root flips sign. The two versions have the same size everywhere; they differ only in sign, on alternate stretches.

In a price. An integral of such a function adds up stretches with the wrong sign. The result is a wrong number, with no error message, and it may even look plausible. Tracking the angle (or rewriting the formula so that the complex number never crosses the negative axis, as Lesson 48 does for Heston) avoids it.

Self-test

1. 1: $e^{i\theta} = \cos\theta + i\sin\theta$, and $\cos^2\theta + \sin^2\theta = 1$. It is the point on the unit circle at angle θ.

2. $e^{-u^2/2}$.

3. Because for them the characteristic function has a formula and the density hasn't. Fourier inversion turns the characteristic function into the probabilities, and so the price.

SOLUTIONS TO LESSON 34

Numerical Methods: Roots, Interpolation, Integration, Fitting

Solutions to Exercises 34.1 to 34.8 and the Lesson 34 self-test.

Exercise 34.1

The idea. Bisection halves the width each step; Newton squares the error each step.

Bisection. After n halvings the width is $0.99/2^n$. We need $0.99/2^n \le 10^{-8}$, so $2^n \ge 0.99 \times 10^8$, that is $n \ge \log_2(0.99 \times 10^8) = 26.56$. So 27 halvings.

Newton. $10^{-2} \to 10^{-4} \to 10^{-8} \to 10^{-16}$: after three steps the error is below 10^{-12}. The number of correct digits doubles each step: 2, 4, 8, 16.

Exercise 34.2

The idea. Replace the function by its tangent line, and find where the tangent crosses zero.

The update. By Taylor's theorem, $C(\sigma_n + \delta) \approx C(\sigma_n) + \mathcal{V}(\sigma_n)\,\delta$. Setting this equal to C^* gives $\delta = -\big(C(\sigma_n) - C^*\big)/\mathcal{V}(\sigma_n)$, and $\sigma_{n+1} = \sigma_n + \delta$.

Why it fails. The step divides by vega. Deep out of the money and at a low volatility, the option has almost no chance of paying off, and its price hardly responds to σ: vega is close to zero (the lesson's case: about 10^{-37}). Then the tangent is nearly flat, it crosses zero very far away, and the next iterate is absurd. Newton's speed comes from trusting the tangent, and here the tangent is useless.

Brent instead. It keeps a bracket $[a, b]$ in which the price error changes sign, so the root can never escape. It tries a fast interpolation step, and accepts it only if it stays inside the bracket and shrinks it enough; otherwise it bisects. So it is never worse than bisection, and usually nearly as fast as Newton.

Exercise 34.3

The idea. On each interval the second derivative of a cubic is a straight line. Integrate twice, fix the constants with the two end values, then match slopes at the knots.

Step 1. The second derivative. On $[t_i, t_{i+1}]$, S'' is linear, from M_i to M_{i+1}:

$$S''(t) = M_i \frac{t_{i+1} - t}{h_i} + M_{i+1} \frac{t - t_i}{h_i}.$$

Step 2. Integrate twice, and require $S(t_i) = y_i$ and $S(t_{i+1}) = y_{i+1}$:

$$\begin{aligned} S(t) = {} & M_i \frac{(t_{i+1} - t)^3}{6h_i} + M_{i+1} \frac{(t - t_i)^3}{6h_i} \\ & + \Big(\frac{y_i}{h_i} - \frac{M_i h_i}{6} \Big)(t_{i+1} - t) + \Big(\frac{y_{i+1}}{h_i} - \frac{M_{i+1} h_i}{6} \Big)(t - t_i). \end{aligned}$$

Step 3. The slope at each end. Differentiating and evaluating:

$$S'(t_i^+) = \frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{3} M_i - \frac{h_i}{6} M_{i+1}, \qquad S'(t_{i+1}^-) = \frac{y_{i+1} - y_i}{h_i} + \frac{h_i}{6} M_i + \frac{h_i}{3} M_{i+1}.$$

Step 4. Match the slopes at t_i. The slope arriving from the interval on the left, $\frac{y_i - y_{i-1}}{h_{i-1}} + \frac{h_{i-1}}{6} M_{i-1} + \frac{h_{i-1}}{3} M_i$, must equal the slope leaving to the right, $\frac{y_{i+1} - y_i}{h_i} - \frac{h_i}{3} M_i - \frac{h_i}{6} M_{i+1}$. Multiplying by 6 and collecting the M's:

$$h_{i-1} M_{i-1} + 2(h_{i-1} + h_i) M_i + h_i M_{i+1} = 6 \Big(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \Big).$$

Values and second derivatives are continuous by construction; this equation makes the slope continuous too. With $M_0 = M_n = 0$ (the natural ends), it is a tridiagonal system.

Exercise 34.4

The idea. Between the points, straight-line zero rates have a constant slope, and the slope changes at each point.

Before 2 years. On $[1, 2]$, $z' = (3.3 - 3.0)/1 = 0.3\%$ per year, so $f(2^-) = 3.3 + 2 \times 0.3 = 3.9\%$.

After 2 years. On $[2, 3]$, $z' = 0.2\%$ per year, so $f(2^+) = 3.3 + 2 \times 0.2 = 3.7\%$.

Why it matters. A forward rate that drops by 0.2% at an arbitrary date says the market expects a sudden fall in rates exactly on that day, which nobody believes. Contracts that depend on rates just after 2 years are priced differently from those just before, and their sensitivities to the quoted rates jump around as quotes move. Prices of anything built on forward rates inherit the saw.

Exercise 34.5

The idea. Check each power directly.

$f(x)$	$\int_{-1}^{1} f\,dx$	$f(-1/\sqrt{3}) + f(1/\sqrt{3})$
1	2	2
x	0	0
x^2	$2/3$	$1/3 + 1/3 = 2/3$
x^3	0	0
x^4	$2/5$	$1/9 + 1/9 = 2/9$

Exact up to x^3; wrong at x^4.

Why degree $2n-1$. A rule with n points has $2n$ numbers to choose: n positions and n weights. Being exact for $1, x, \ldots, x^{2n-1}$ is $2n$ conditions. Gauss showed the conditions can always be met, with all positions inside the interval and all weights positive.

Exercise 34.6

The idea. A minimum of a smooth function has zero derivative.

The normal equations. $\|y - X\beta\|^2 = (y - X\beta)^\top (y - X\beta)$. Its derivative with respect to β is $-2X^\top(y - X\beta)$. Setting it to zero gives $X^\top X\beta = X^\top y$.

The line. With $\bar{x} = 1$ and $\bar{y} = 5/3$, the slope is

$$b = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{(-1)(-\frac{2}{3}) + 0 + (1)(\frac{1}{3})}{2} = 0.5,$$

and $a = \bar{y} - b\bar{x} = 5/3 - 1/2 = 7/6 \approx 1.1667$. The fitted values $7/6, 5/3, 13/6$ leave errors $-1/6, 1/3, -1/6$, which add up to zero and are at right angles to $(0, 1, 2)$: the normal equations, checked.

Exercise 34.7

A safeguarded implied-volatility solver, against plain Newton.

```
import numpy as np


def bs_call(s, k, t, vol):
    d1 = (math.log(s / k) + (r + 0.5 * vol**2) * t) / (vol * math.sqrt(t))
    return s * N(d1) - k * math.exp(-r * t) * N(d1 - vol * math.sqrt(t))


def bs_vega(s, k, t, vol):
    d1 = (math.log(s / k) + (r + 0.5 * vol**2) * t) / (vol * math.sqrt(t))
    return s * math.exp(-d1 * d1 / 2) / math.sqrt(2 * math.pi) * math.sqrt(t)


def implied_vol(price, k, t, lo=1e-4, hi=5.0, tol=1e-12):
    # Newton from the middle of a bracket; any step that leaves the bracket becomes a bisec-
tion step.
    f = lambda v: bs_call(S0, k, t, v) - price
    flo = f(lo)
    x = 0.3
    for _ in range(100):
        fx = f(x)
        if abs(fx) < tol:
            return x
        if (fx > 0) == (flo > 0):          # keep the bracket [lo, hi] around the root
            lo, flo = x, fx
        else:
            hi = x
        vg = bs_vega(S0, k, t, x)
        step = x - fx / vg if vg > 1e-12 else None
        x = step if step is not None and lo < step < hi else 0.5 * (lo + hi)
    return x


def plain_newton(price, k, t, x=0.2):
    for _ in range(50):
        vg = bs_vega(S0, k, t, x)
        if not (vg > 1e-12) or not math.isfinite(x) or x <= 0 or x > 50:
            return None
        x = x - (bs_call(S0, k, t, x) - price) / vg
    return x


rng = np.random.default_rng(347)
worst, fails, cases, skipped = 0.0, 0, 0, 0
for k in np.linspace(50, 200, 20):
    for t in np.geomspace(0.1, 5, 10):
        v_true = float(rng.uniform(0.10, 0.60))
        price = bs_call(S0, float(k), float(t), v_true)
        if price - max(S0 - k * math.exp(-r * t), 0) < 1e-6:    # no time value: the price
says nothing about sigma
            skipped += 1
            continue
        cases += 1
        worst = max(worst, abs(implied_vol(price, float(k), float(t)) - v_true))
```

```
        pn = plain_newton(price, float(k), float(t))
        fails += pn is None or abs(pn - v_true) > 1e-6
print(f"cases {cases} (skipped {skipped} with time value below 1e-6), largest error of the
safeguarded solver {worst:.1e}")
print(f"plain Newton from 20% failed in {fails} cases")
```

What it prints.

```
cases 187 (skipped 13 with time value below 1e-6), largest error of the safeguarded solver
6.6e-10
plain Newton from 20% failed in 51 cases
```

Reading it. 13 of the 200 cases are skipped: options so deep in the money, at such short maturities, that their price is the intrinsic value to within 10^{-6}. Their price carries no information about σ, and no solver could recover it. On the other 187, the safeguarded solver is right to 6.6e-10. Plain Newton from 20% fails in 51 of them: wherever vega at 20% is small, its first step overshoots, and it never finds its way back.

Exercise 34.8

Levenberg-Marquardt from three starts, with and without a bound, then a grid over τ.

```
tm = np.array(o["fit"]["t"])
yld = np.array(o["fit"]["y_pct"]) / 100
g = lambda t, tau: (1 - np.exp(-t / tau)) / (t / tau)
model = lambda p: p[0] + p[1] * g(tm, p[2])


def jac(p):
    x = tm / p[2]
    dg = (np.exp(-x) * (1 + x) - 1) / x**2
    return np.column_stack([np.ones_like(tm), g(tm, p[2]), p[1] * dg * (-tm / p[2] ** 2)])


def lm(p0, tau_min=None, iters=60):
    p = np.array(p0, float); lam = 1e-2
    for _ in range(iters):
        res = yld - model(p); cur = float(res @ res)
        J = jac(p); A = J.T @ J
        if np.linalg.matrix_rank(A) < 3:
            return p, cur, "the Jacobian has a zero column: no information about tau"
        while lam < 1e12:
            pn = p + np.linalg.solve(A + lam * np.diag(np.diag(A)), J.T @ res)
            if tau_min is not None:
                pn[2] = max(pn[2], tau_min)
            if pn[2] > 0 and float((yld - model(pn)) @ (yld - model(pn))) < cur:
                p, lam = pn, lam / 3
                break
            lam *= 3
        else:
            break
    res = yld - model(p)
    return p, float(res @ res), "ok"


for bound in (None, 0.05):
    print("tau >= 0.05" if bound else "no bound")
    for start in ((0.05, -0.03, 20.0), (0.03, 0.01, 0.5), (0.04, 0.0, 2.0)):
        p, sse, note = lm(start, bound)
        print(f"  start {start}:  b0 {p[0]:.5f}  b1 {p[1]:+.5f}  tau {p[2]:8.4f}  SSE
{sse:.2e}  {note}")

# What does work: for each tau on a grid, b0 and b1 enter linearly, so solve for them by
least squares;
# keep the best tau, then polish all three with Levenberg-Marquardt.
best = None
for tau in np.geomspace(0.1, 30, 60):
    X = np.column_stack([np.ones_like(tm), g(tm, tau)])
    beta = np.linalg.lstsq(X, yld, rcond=None)[0]
    res = yld - X @ beta
    if best is None or res @ res < best[0]:
        best = (float(res @ res), beta[0], beta[1], tau)
p, sse, note = lm(best[1:])
print(f"grid over tau, then LM: start tau {best[3]:.3f} -> b0 {p[0]:.5f}  b1 {p[1]:+.5f}  tau
{p[2]:.4f}  SSE {sse:.2e}")
```

What it prints.

```
no bound
  start (0.05, -0.03, 20.0):  b0 0.04200  b1 -0.01389  tau   2.0280  SSE 4.64e-08  ok
  start (0.03, 0.01, 0.5):  b0 0.03612  b1 +0.00006  tau   0.1036  SSE 9.50e-05  the Jacobian
has a zero column: no information about tau
  start (0.04, 0.0, 2.0):  b0 0.04000  b1 +0.00000  tau   2.0000  SSE 1.70e-04  the Jacobian
has a zero column: no information about tau
tau >= 0.05
  start (0.05, -0.03, 20.0):  b0 0.04200  b1 -0.01389  tau   2.0280  SSE 4.64e-08  ok
  start (0.03, 0.01, 0.5):  b0 0.04064  b1 -0.00056  tau   0.0500  SSE 2.05e-04  the Jacobian
has a zero column: no information about tau
  start (0.04, 0.0, 2.0):  b0 0.04000  b1 +0.00000  tau   2.0000  SSE 1.70e-04  the Jacobian
has a zero column: no information about tau
grid over tau, then LM: start tau 2.002 -> b0 0.04200  b1 -0.01389  tau 2.0280  SSE 4.64e-08
```

Reading it. The first start succeeds. The other two fail for the same reason: they end with $b_1 \approx 0$. When $b_1 = 0$, the model is the flat line b_0, and τ has no effect on it at all: the Jacobian's column for τ is zero, and no step can say which way to move it. The third start begins there; the second drifts there. The bound $\tau \geq 0.05$ keeps τ positive but doesn't fix the cause.

What works. For each fixed τ, the model is linear in b_0 and b_1, so they can be found exactly by least squares. Scan τ over a grid, keep the best, and start Levenberg-Marquardt from there: it reaches the right answer, with τ near 2.00 on the grid and an error of 4.6e-08 after polishing. Splitting off the parameters that enter linearly is a standard way to make a nonlinear fit robust.

Self-test

1. Newton uses the slope (vega) to jump straight to where the tangent crosses zero, so its error squares each step; bisection only halves a bracket. Newton needs the derivative and a good starting point; bisection needs only a bracket.

2. Because a forward rate is $z + t\,z'$: it depends on the slope of the zero curve, magnified by t. Small kinks or wiggles in the zero curve become jumps and humps in the forwards, and the forwards are what price future cash flows.

3. As λ grows, the step gets shorter and points straight downhill. λ is divided by 3 after each step that lowers the error, moving the method towards Gauss-Newton, and multiplied by 3 after each step that doesn't.

SOLUTIONS TO LESSON 35

Regression and PDEs, Further

Solutions to Exercises 35.1 to 35.8 and the Lesson 35 self-test.

Exercise 35.1

The idea. Split the error into the part no function of X can remove, plus the part that g controls.

Step 1. Write $m(X) = E[Y \mid X]$. Then

$$\big(Y - g(X)\big)^2 = \big(Y - m(X)\big)^2 + 2\big(Y - m(X)\big)\big(m(X) - g(X)\big) + \big(m(X) - g(X)\big)^2.$$

Step 2. The cross term has mean zero. Condition on X: the factor $m(X) - g(X)$ is known, and comes out (Lesson 8), leaving $E[Y - m(X) \mid X] = m(X) - m(X) = 0$. By the tower property its average is zero too.

Step 3. So $E\big[(Y - g(X))^2\big] = E\big[(Y - m(X))^2\big] + E\big[(m(X) - g(X))^2\big]$. The first term doesn't depend on g, and the second is never negative and is zero when $g = m$. ■

Exercise 35.2

The idea. The regression is only used to decide whether to exercise, and that question only arises in the money.

Why only in the money. Out of the money, exercising is worth nothing, so the continuation value is never compared with anything there. Its estimate only has to be good where the put is in the money.

What goes wrong with all paths. A few basis functions have to describe the continuation value over the whole range of share prices. Above the strike, the continuation value flattens towards zero; below, it rises steeply. A low-degree polynomial can't follow both, and the least-squares fit compromises: it bends to follow the flat region, and is worse in the money, exactly where exercise is decided. In the lesson's test, the straight line's error in the money was 2.68 when fitted to all paths and 0.75 when fitted only in the money. A worse continuation value means a worse exercise rule, and a lower price.

Exercise 35.3

The idea. A weighted average, with the weights from the kernel.

Step 1. The weights. With $h = 10$: at $X = 90$ and 110, $K_h = e^{-\frac{1}{2}(10/10)^2} = e^{-1/2} = 0.6065$; at $X = 100$, $K_h = 1$.

Step 2. The estimate.

$$\hat{m}(100) = \frac{0.6065 \times 10 + 1 \times 0 + 0.6065 \times 2}{0.6065 + 1 + 0.6065} = \frac{7.2784}{2.2131} = 3.2888.$$

The limits. As $h \to 0$, the weight at 100 dominates and $\hat{m}(100) \to 0$: the estimate is that one point's own Y, all noise. As $h \to \infty$, all weights tend to 1 and $\hat{m}(100) \to (10 + 0 + 2)/3 = 4$: the overall average, which ignores X completely.

Choosing h. Neither extreme estimates the conditional expectation. The bandwidth must be large enough to average out the noise and small enough that the points it averages have similar conditional expectations.

Exercise 35.4

The idea. Differentiate the log of the density, and compare both sides.

Step 1. Write $z = x - x_0 - at$, so that $p = (2\pi b^2 t)^{-1/2} e^{-z^2/(2b^2 t)}$. Note $\partial z/\partial x = 1$ and $\partial z/\partial t = -a$.

Step 2. The space derivatives.

$$p_x = -\frac{z}{b^2 t}\, p, \qquad p_{xx} = \Big(\frac{z^2}{b^4 t^2} - \frac{1}{b^2 t}\Big) p.$$

Step 3. The time derivative. Differentiating $\ln p = -\frac{1}{2}\ln(2\pi b^2 t) - z^2/(2b^2 t)$ with respect to t:

$$\frac{p_t}{p} = -\frac{1}{2t} + \frac{az}{b^2 t} + \frac{z^2}{2b^2 t^2}.$$

Step 4. The right-hand side.

$$\frac{-a\, p_x + \frac{1}{2} b^2 p_{xx}}{p} = \frac{az}{b^2 t} + \frac{z^2}{2b^2 t^2} - \frac{1}{2t}.$$

The two agree. ✓

Exercise 35.5

The idea. Integrate the forward equation, and move derivatives across by integrating by parts. Far out, p and its derivatives vanish, so all boundary terms do too.

Total probability.

$$\frac{d}{dt}\int p\,dx = \int \Big(-(ap)_x + \tfrac{1}{2}(b^2 p)_{xx}\Big)dx = \Big[-ap + \tfrac{1}{2}(b^2 p)_x\Big]_{-\infty}^{\infty} = 0.$$

The pairing.

$$\begin{aligned}\frac{d}{dt}\int g\,p\,dx &= \int g_t\,p\,dx + \int g\Big(-(ap)_x + \tfrac{1}{2}(b^2 p)_{xx}\Big)dx \\ &= \int \Big(g_t + a\,g_x + \tfrac{1}{2}b^2 g_{xx}\Big)p\,dx = 0,\end{aligned}$$

moving one derivative across for the first term and two for the second.

Why that makes them agree. At $t = 0$, p is a spike at today's value x_0, so $\int g\,p\,dx = g(0, x_0)$: the backward answer. At $t = T$, $g(T, x) = h(x)$, so $\int g\,p\,dx = \int h(x)\,p(T, x)\,dx$: the forward answer. The quantity doesn't change in between, so the two are equal.

Exercise 35.6

The idea. As in Lesson 29, but for two variables.

Step 1. The derivatives. With $V(t, S_1, S_2) = U(\tau, x, y)$: $S_1 V_1 = U_x$, $S_1^2 V_{11} = U_{xx} - U_x$, $S_2 V_2 = U_y$, $S_2^2 V_{22} = U_{yy} - U_y$, and $S_1 S_2 V_{12} = U_{xy}$ (differentiate U_x / S_1 with respect to S_2). And $V_t = -U_\tau$.

Step 2. Substitute.

$$-U_\tau + \tfrac{1}{2}\sigma_1^2(U_{xx} - U_x) + \tfrac{1}{2}\sigma_2^2(U_{yy} - U_y) + \rho\sigma_1\sigma_2 U_{xy} + rU_x + rU_y = rU.$$

Collecting the U_x and U_y terms gives the lesson's equation, with $\nu_i = r - \frac{1}{2}\sigma_i^2$. The coefficients are constants, as in one dimension.

The work per step, with $M = 100$:

- **Full implicit:** M^2 unknowns in a banded system of bandwidth M. Its factorisation costs about $M^2 \times M^2 = M^4 = 100{,}000{,}000$ operations.
- **ADI:** two sweeps, each M tridiagonal systems of size M at about $8M$ operations: roughly $16M^2 = 160{,}000$.

About 600 times less work, and the gap widens as the grid gets finer.

Exercise 35.7

Longstaff-Schwartz with polynomial and kernel regressions.

```
import numpy as np

dates, paths = 50, 50_000
dt = T / dates
df = math.exp(-r * dt)
rng = np.random.default_rng(357)
S = S0 * np.exp(((r - 0.5 * sigma**2) * dt + sigma * math.sqrt(dt) *
rng.standard_normal((paths, dates))).cumsum(axis=1))


def lsm(regress):
    cash = np.maximum(K - S[:, -1], 0.0)
    for i in range(dates - 2, -1, -1):
        cash *= df
        itm = K - S[:, i] > 0
        cont = regress(S[itm, i], cash[itm])
        ex = K - S[itm, i]
        do = ex > cont
        cash[np.where(itm)[0][do]] = ex[do]
    return float((cash * df).mean())


def poly3(x, y):
    return np.polyval(np.polyfit(x / K, y, 3), x / K)


def kernel(h):
    def fit(x, y):
        grid = np.linspace(x.min(), x.max(), 120)              # estimate on a grid, then in-
terpolate
        wts = np.exp(-0.5 * ((grid[:, None] - x[None, :]) / h) ** 2)
        m = (wts @ y) / wts.sum(axis=1)
        return np.interp(x, grid, m)
    return fit


print("regression            American put")
res = {"poly": lsm(poly3)}
print(f"cubic polynomial      {res['poly']:.4f}")
for h in (0.5, 1.0, 2.0, 5.0, 15.0):
    res[h] = lsm(kernel(h))
    print(f"kernel, h = {h:4.1f}      {res[h]:.4f}")
print("tree (Lesson 22): about 9.478")
```

What it prints.

```
regression            American put
cubic polynomial      9.4540
kernel, h =  0.5      9.5209
kernel, h =  1.0      9.4600
kernel, h =  2.0      9.2886
kernel, h =  5.0      8.6914
kernel, h = 15.0      8.5087
tree (Lesson 22): about 9.478
```

Reading it. The answer depends strongly on the bandwidth: from 9.5209 at $h = 0.5$ to 8.5087 at $h = 15$. A small bandwidth fits each path's own future cash flow: the rule is fitted and valued on the same paths, so it gains a little foresight, and the price is biased *up*, here above the tree's 9.478 (Lesson 30, Exercise 30.6). A large bandwidth flattens the continuation value and blurs the exercise boundary: the rule is poor, and the price is biased *down*. At $h = 1$, 9.4600, it is close to the cubic polynomial's 9.4540. A kernel regression needs its bandwidth chosen with care, ideally by valuing on fresh paths; a few polynomial terms are far less fussy.

Exercise 35.8

Craig-Sneyd for the basket call, checked by Monte Carlo.

```
a1, a2, s1, s2, rho = 100.0, 95.0, 0.20, 0.30, 0.5


def thomas(a, b, c, d):
    n = len(b); cp = np.zeros(n); dp = np.zeros_like(d, dtype=float)
    cp[0] = c[0] / b[0]; dp[0] = d[0] / b[0]
    for i in range(1, n):
        m = b[i] - a[i] * cp[i - 1]
        cp[i] = c[i] / m if i < n - 1 else 0.0
        dp[i] = (d[i] - a[i] * dp[i - 1]) / m
    x = np.zeros_like(dp); x[-1] = dp[-1]
    for i in range(n - 2, -1, -1):
        x[i] = dp[i] - cp[i] * x[i + 1]
    return x


def craig_sneyd(h, payoff, edge_value, half=1.5, damp=4):
    n_half = math.ceil(half / h)
    x = math.log(a1) + h * np.arange(-n_half, n_half + 1)
    y = math.log(a2) + h * np.arange(-n_half, n_half + 1)
    n = len(x); N_ = n - 1; k = T / N_
    nu1, nu2 = r - 0.5 * s1**2, r - 0.5 * s2**2
    X, Y = np.meshgrid(np.exp(x), np.exp(y), indexing="ij")
    U = payoff(X, Y)
    A1 = lambda V: np.pad(nu1 * (V[2:] - V[:-2]) / (2 * h) + 0.5 * s1**2 * (V[2:] - 2 *
V[1:-1] + V[:-2]) / h**2 - 0.5 * r * V[1:-1], ((1, 1), (0, 0)))
    A2 = lambda V: np.pad(nu2 * (V[:, 2:] - V[:, :-2]) / (2 * h) + 0.5 * s2**2 * (V[:, 2:] -
2 * V[:, 1:-1] + V[:, :-2]) / h**2 - 0.5 * r * V[:, 1:-1], ((0, 0), (1, 1)))
    A0 = lambda V: np.pad(rho * s1 * s2 * (V[2:, 2:] - V[2:, :-2] - V[:-2, 2:] + V[:-2,
:-2]) / (4 * h * h), ((1, 1), (1, 1)))

    def solve(nu_, s_, rhs, th):
        lo, mi, up = 0.5 * s_**2 / h**2 - nu_ / (2 * h), -s_**2 / h**2 - 0.5 * r, 0.5 *
s_**2 / h**2 + nu_ / (2 * h)
        a = np.full(n, -th * k * lo); b = np.full(n, 1 - th * k * mi); c = np.full(n, -th *
k * up)
        a[0] = c[0] = a[-1] = c[-1] = 0.0; b[0] = b[-1] = 1.0
        return thomas(a, b, c, rhs)

    def sweeps(Y0, a1U, a2U, th, edge):
        Y1 = solve(nu1, s1, Y0 - th * k * a1U, th); Y1[0], Y1[-1] = edge[0], edge[-1]
        Y2 = solve(nu2, s2, (Y1 - th * k * a2U).T, th).T
        Y2[:, 0], Y2[:, -1], Y2[0], Y2[-1] = edge[:, 0], edge[:, -1], edge[0], edge[-1]
        return Y2

    for step in range(N_):
        th = 1.0 if step < damp else 0.5
        edge = edge_value(X, Y, (step + 1) * k)
        a0U, a1U, a2U = A0(U), A1(U), A2(U)
        Y0 = U + k * (a0U + a1U + a2U)
        U2 = sweeps(Y0, a1U, a2U, th, edge)
        U = sweeps(Y0 + 0.5 * k * (A0(U2) - a0U), a1U, a2U, th, edge)
    return float(U[n_half, n_half]), n
```

```
basket = lambda X, Y: np.maximum(0.5 * (X + Y) - 100, 0)
basket_edge = lambda X, Y, tau: np.maximum(0.5 * (X + Y) - 100 * math.exp(-r * tau), 0)   #
deep in or out of the money
for h in (0.05, 0.025, 0.0125):
    v, n = craig_sneyd(h, basket, basket_edge)
    print(f"Craig-Sneyd, h = {h:.4f} ({n} x {n} grid): {v:.4f}")

rng = np.random.default_rng(358)
Z = rng.standard_normal((2_000_000, 2))
Z2 = rho * Z[:, 0] + math.sqrt(1 - rho**2) * Z[:, 1]                # Lesson 31's Cholesky, 2
x 2
ST1 = a1 * np.exp((r - 0.5 * s1**2) * T + s1 * math.sqrt(T) * Z[:, 0])
ST2 = a2 * np.exp((r - 0.5 * s2**2) * T + s2 * math.sqrt(T) * Z2)
pay = math.exp(-r * T) * np.maximum(0.5 * (ST1 + ST2) - 100, 0)
mc, se = float(pay.mean()), float(pay.std() / math.sqrt(len(pay)))
print(f"Monte Carlo, 2,000,000 paths: {mc:.4f} (standard error {se:.4f})")
```

What it prints.

```
Craig-Sneyd, h = 0.0500 (61 x 61 grid): 8.6045
Craig-Sneyd, h = 0.0250 (121 x 121 grid): 8.6136
Craig-Sneyd, h = 0.0125 (241 x 241 grid): 8.6157
Monte Carlo, 2,000,000 paths: 8.6110 (standard error 0.0101)
```

Reading it. The grid prices settle at 8.6157, and the simulation gives 8.6110 with a standard error of 0.0101: they agree well within one standard error. Two details from the code: the boundary values use the discounted strike, $\max\left(\frac{1}{2}(S_1 + S_2) - 100e^{-r\tau}, 0\right)$, which is the right value far in or far out of the money; and the payoff's kink, the line $S_1 + S_2 = 200$, doesn't fall on grid points, which makes the convergence less regular than for Margrabe, but still steady.

Self-test

1. The conditional expectation of the discounted payoff given today's share price: the continuation value, estimated within the family of basis functions used.

2. Forwards in time, from today. It carries the density of the share (or its log) from today's spike to its spread-out shape at later times.

3. Each implicit correction links grid points along one direction only, so it is a batch of tridiagonal systems, solved in about M^2 operations in all, as for an explicit step. Craig-Sneyd adds one stage that recomputes the cross term with the new values and repeats the two corrections, which raises the accuracy from first to second order in time.